SAT encodings for empty-board N-Queens: a comparison with ILP and CP

Vu Minh Hoang[a,*]

[a]VNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens Problem
SAT Encoding
At-Most-One Constraint
Combinatorial Optimization
Integer Linear Programming
Constraint Programming
CPLEX
Gurobi
CP-SAT

ABSTRACT

The choice of at-most-one (AMO) encoding changes the size and propagation behavior of a Boolean satisfiability (SAT) formula for N-Queens. This study implements pairwise, binary, commander, sequential counter, and product encodings and compares them with integer linear programming (ILP) and constraint programming (CP) baselines on empty boards. The comparison covers ten methods at $N \in \{20, 50, 100, 150, 200\}$, with one solver worker, three repetitions, and a 120-s full-process deadline per run. Exact formula counts, projected AMO semantics, and unit propagation are checked independently. At $N = 200$, Commander/Glucose4 has the lowest SAT median at 2.252 s; CP Optimizer records 0.064 s, Gurobi 0.474 s, and CPLEX MIP 1.467 s. Sequential and CBC reach the deadline in all three repetitions. Gurobi environment initialization is measured separately. Measurements from separate sessions retain their provenance and are compared descriptively. One machine and five feasible board sizes do not establish general solver superiority.

1. Introduction

The N-Queens problem asks for N non-attacking queens on an $N \times N$ board. Direct constructions find a placement for $N = 1$ and every $N \geq 4$ in polynomial time [3]. The empty-board problem still permits a controlled comparison of SAT, integer linear programming (ILP), and constraint programming (CP) formulations. Its results do not extend automatically to N-Queens Completion or unrelated combinatorial problems [6].

SAT solvers use conflict-driven clause learning (CDCL), but they require a conjunctive normal form (CNF) representation of the placement constraints [12]. Different at-most-one (AMO) encodings can change both CNF size and what unit propagation deduces [17]. A comparison requires the same attempted board sizes for every method, including explicit timeout outcomes.

This study asks how the five encodings differ in formula size and measured runtime under a common process limit. It also compares SAT with CBC, Gurobi, CPLEX MIP, OR-Tools CP-SAT, and CP Optimizer. The paper checks exact CNF counts and AMO propagation and retains the provenance of each measurement session. The study does not propose a new encoding or claim a general solver ranking.

2. Background and related work

Empty boards with $N = 2$ or $N = 3$ have no solution; the present task seeks one placement for feasible sizes. N-Queens Completion adds pre-placed queens and has NP-complete decision and #P-complete counting versions [6]. Those harder variants are outside this experiment.

SAT asks whether a Boolean formula has a satisfying assignment. A CNF formula is a conjunction of clauses, each a disjunction of literals. Modern solvers extend DPLL with conflict-driven clause learning (CDCL) [12, 13, 15]. Glucose uses literal block distance (LBD), the number of decision levels in a learned clause, to assess learned clauses [1, 2]. The SAT experiments use Glucose4 through PySAT [10].

For inputs $X = \{x_1, \ldots, x_n\}$, at least one (ALO) is the clause $\bigvee_i x_i$, at most one (AMO) requires $\sum_i x_i \leq 1$, and exactly one (EXO) combines ALO and AMO. Established AMO encodings trade auxiliary variables and clause counts against propagation cost [5, 17]. Sinz introduced sequential counters [20], Klieber and Kwon proposed commander groups [11], and Chen studied product encodings [4].

*Corresponding author
24021494@vnu.edu.vn (V.M. Hoang)

ILP combines linear constraints with integer variables and can use branch-and-bound and cuts [16]. The ILP baselines are CBC through PuLP, Gurobi, and CPLEX MIP [8, 9, 14]. CP uses finite-domain constraints and may exploit specialized filtering for `AllDifferent`, such as Régin's matching method [18, 19]. The CP baselines are OR-Tools CP-SAT and IBM CP Optimizer [7, 9]; their internal propagation cannot be inferred from the model alone.

3. SAT encodings for N-Queens

3.1. Problem formulation

For each cell (i, j) of an $N \times N$ board, let $x_{i,j}$ be true exactly when that cell contains a queen. Here $i, j \in \{1, \dots, N\}$.

The board-level constraint has six components. For each encoding E, auxiliary variables Y are allocated separately for each AMO set, and the resulting CNF $\Phi_E(X, Y)$ satisfies $\Phi_{\text{board}}(X) \iff \exists Y\, \Phi_E(X, Y)$:

$$\Phi = \Phi^{row}_{ALO} \wedge \Phi^{row}_{AMO} \wedge \Phi^{col}_{ALO} \wedge \Phi^{col}_{AMO} \wedge \Phi^{diag}_{AMO} \wedge \Phi^{anti}_{AMO} \tag{1}$$

3.2. Row and column constraints

Every row and column must contain exactly one queen. For row i, the ALO clause is

$$\bigvee_{j=1}^{N} x_{i,j} \tag{2}$$

The chosen encoding supplies row AMO clauses; columns use the same construction. Each row and column therefore satisfies EXO = ALO ∧ AMO.

3.3. Diagonal constraints

Cells share a main diagonal when $d = i - j$ is equal, with $d \in \{-(N-1), \dots, N-1\}$. They share an anti-diagonal when $s = i + j$ is equal, with $s \in \{2, \dots, 2N\}$. Each diagonal needs AMO but may be empty, so it needs no ALO clause.

For example, at $N = 4$ the row-wise column sequence $(2, 4, 1, 3)$ gives the placement

$$X = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Every row and column sums to one, and every diagonal sums to at most one. The implementation numbers a cell by $v(i, j) = N(i-1) + j$ for the one-based indices used here; the true board literals in this example are $2, 8, 9, 15$. Auxiliary IDs start after N^2. Decoding reads only board literals, and validation checks the coordinates independently of the encoding.

3.4. AMO encoding schemes

For n Boolean variables $X = \{x_1, \dots, x_n\}$, AMO permits at most one true input:

$$\sum_{i=1}^{n} x_i \leq 1 \tag{3}$$

The five implementations encode this constraint in CNF with different auxiliary-variable and clause counts. For $n \leq 1$, AMO requires no clauses or auxiliary variables; formulas below that use counters or logarithms assume $n \geq 2$. All five propagate a true input to the falsity of every other AMO input.

3.4.1. Binomial (pairwise) encoding

The pairwise encoding forbids every pair of inputs from being true together:

$$\Phi_{\text{binomial}} = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \tag{4}$$

It uses no auxiliary variables and $\binom{n}{2} = O(n^2)$ binary clauses per AMO constraint. Across the board, the clause count is $O(N^3)$. If x_i is true, unit propagation falsifies every other input x_j. This is generalized arc consistency (GAC) on the original AMO inputs.

3.4.2. Binary encoding

Binary encoding assigns each input a code over $m = \lceil \log_2 n \rceil$ auxiliary bits $Y = \{y_0, \dots, y_{m-1}\}$ [5]. Input x_{k+1} has index $k \in \{0, \dots, n-1\}$ and a unique expansion $k = \sum_{b=0}^{m-1} \mathrm{bit}(k, b) 2^b$.

The literal required by bit b of code k is

$$\ell(k, b) = \begin{cases} y_b & \text{if bit } b \text{ of } k \text{ is 1,} \\ \neg y_b & \text{if bit } b \text{ of } k \text{ is 0.} \end{cases} \tag{5}$$

The binary AMO formula is

$$\Phi_{\text{binary}} = \bigwedge_{k=0}^{n-1} \bigwedge_{b=0}^{m-1} (\neg x_{k+1} \vee \ell(k, b)) \tag{6}$$

If x_{k+1} is true, unit propagation fixes every bit in Y. Each other input requires a different bit in at least one position, so its corresponding clause propagates $\neg x_j$. This establishes GAC on the original AMO inputs, not propagation completeness for arbitrary auxiliary assignments. The encoding uses $\lceil \log_2 n \rceil$ auxiliary variables and $n\lceil \log_2 n \rceil$ binary clauses per constraint.

3.4.3. Commander encoding

Commander encoding partitions X into groups $G_1, \dots, G_m$ of at most three inputs and applies the construction recursively [11]. Each group has one auxiliary commander variable c_k.

The encoding has four clause sets:

1. **Within-group AMO (Φ_{intra}):** Forbid two true inputs in one group:

$$\Phi_{\text{intra}} = \bigwedge_{k=1}^{m} \bigwedge_{\substack{x_i, x_j \in G_k \\ i<j}} (\neg x_i \vee \neg x_j) \tag{7}$$

2. **Forward implication (Φ_{fwd}):** A true group input forces its commander true:

$$\Phi_{\text{fwd}} = \bigwedge_{k=1}^{m} \bigwedge_{x \in G_k} (\neg x \vee c_k) \tag{8}$$

3. **Reverse implication (Φ_{rev}):** A true commander requires a true group input:

$$\Phi_{\text{rev}} = \bigwedge_{k=1}^{m} \left(\neg c_k \vee \bigvee_{x \in G_k} x \right) \tag{9}$$

4. **Commander AMO (Φ_{cmd}):** Apply AMO recursively to the commanders:

$$\Phi_{\text{cmd}} = \mathrm{AMO}(\{c_1, c_2, \dots, c_m\}) \tag{10}$$

The recursion terminates at $n \leq 3$ with pairwise AMO. The implementation includes reverse implications, so $c_k \leftrightarrow \bigvee_{x \in G_k} x$. Reverse clauses are optional for the projected AMO relation and input GAC. Forward implications, within-group AMO, and recursive commander AMO already propagate $\neg x_j$ after x_i becomes true [11]. The implemented version uses $O(n)$ auxiliary variables and clauses.

3.4.4. Sequential counter encoding

Sinz's sequential counter uses $n-1$ auxiliary variables $s_1, \dots, s_{n-1}$ [20]. They carry the prefix implications needed to reject a second true input. The clauses do not force each s_i to equal the prefix disjunction in every satisfying auxiliary assignment.

The clauses enforce prefix propagation and collision prohibition:

$$(\neg x_1 \vee s_1) \tag{11}$$

$$\bigwedge_{i=2}^{n-1} \left((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1}) \right) \tag{12}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{13}$$

The encoding uses $n - 1 = O(n)$ auxiliary variables and $3n - 4 = O(n)$ clauses per constraint. If x_i is true, unit propagation follows the prefix chain to falsify later inputs and propagates backward to falsify earlier inputs. It is GAC on the original AMO inputs.

3.4.5. Product encoding

Product encoding places the n inputs on a two-dimensional grid [4]. It sets $p = \lceil \sqrt{n} \rceil$ and $q = \lceil n/p \rceil$, so $n \leq pq$. The auxiliary coordinates are $U = \{u_1, \ldots, u_p\}$ and $V = \{v_1, \ldots, v_q\}$.

Each variable x_{k+1} ($k \in \{0, \ldots, n-1\}$) is mapped to grid coordinates $r = \lfloor k/q \rfloor + 1$ and $c = (k \bmod q) + 1$ via channeling clauses:

$$\Phi_{\text{chan}} = \bigwedge_{k=0}^{n-1} \left((\neg x_{k+1} \vee u_{\lfloor k/q \rfloor + 1}) \wedge (\neg x_{k+1} \vee v_{(k \bmod q)+1}) \right) \tag{14}$$

Pairwise AMO constraints are then enforced independently across U and V:

$$\Phi_{\text{grid}} = \bigwedge_{1 \leq i < j \leq p} (\neg u_i \vee \neg u_j) \wedge \bigwedge_{1 \leq i < j \leq q} (\neg v_i \vee \neg v_j) \tag{15}$$

To eliminate invalid phantom cells when $p \times q > n$, phantom clauses forbid vacant coordinate pairs:

$$\Phi_{\text{phantom}} = \bigwedge_{r=1}^{p} \bigwedge_{\substack{c=1 \\ (r-1)q+(c-1) \geq n}}^{q} (\neg u_r \vee \neg v_c) \tag{16}$$

For $n < 6$, the implementation uses pairwise AMO instead of creating grid variables. For $n \geq 6$, a true x_{k+1} fixes its two coordinates. Coordinate AMO then falsifies every other input through at least one differing coordinate. The encoding is GAC on the original AMO inputs and uses $O(\sqrt{n})$ auxiliary variables and $O(n)$ clauses per constraint.

3.5. Baseline exact formulations

3.5.1. Constraint programming formulation (CP)

The CP model uses N integer variables $Q = \{q_1, \ldots, q_N\}$. Variable $q_i \in \{1, \ldots, N\}$ gives the queen's column in row i, so the variable indices enforce row occupancy. Three `AllDifferent` constraints enforce distinct columns and diagonals:

$$\text{AllDifferent}(q_1, q_2, \ldots, q_N) \tag{17}$$

$$\text{AllDifferent}(q_1 + 1, q_2 + 2, \ldots, q_N + N) \tag{18}$$

$$\text{AllDifferent}(q_1 - 1, q_2 - 2, \ldots, q_N - N) \tag{19}$$

Régin's matching algorithm is one possible way to enforce GAC for `AllDifferent` [18]. This model does not establish which propagators or presolve steps either CP solver used in a particular run.

3.5.2. Integer linear programming formulation (ILP)

The ILP model uses N^2 binary variables $x_{i,j}$, where $x_{i,j} = 1$ exactly when cell (i, j) contains a queen. For $N \geq 2$, four families of linear constraints enforce non-attacking placement:

$$\sum_{j=1}^{N} x_{i,j} = 1 \quad \forall i \in \{1, \ldots, N\} \quad \text{(Row constraints)} \tag{20}$$

$$\sum_{i=1}^{N} x_{i,j} = 1 \quad \forall j \in \{1, \ldots, N\} \quad \text{(Column constraints)} \tag{21}$$

Table 1
Asymptotic size of the N-Queens SAT encodings

Encoding	Total variables	Total clauses	Input GAC via UP
Binomial (Pairwise)	N^2	$O(N^3)$	Yes
Binary	$N^2 + O(N \log N)$	$O(N^2 \log N)$	Yes
Commander	$N^2 + O(N^2)$	$O(N^2)$	Yes
Sequential Counter	$N^2 + O(N^2)$	$O(N^2)$	Yes
Product	$N^2 + O(N\sqrt{N})$	$O(N^2)$	Yes

$$\sum_{\substack{1\le i,j\le N \\ i-j=d}} x_{i,j} \le 1 \quad \forall d \in \{-(N-2), \dots, N-2\} \quad \text{(Main diagonals)} \tag{22}$$

$$\sum_{\substack{1\le i,j\le N \\ i+j=s}} x_{i,j} \le 1 \quad \forall s \in \{3, \dots, 2N-1\} \quad \text{(Anti-diagonals)} \tag{23}$$

Singleton diagonals are omitted because their inequalities are redundant. For $N \ge 2$, each ILP model therefore has N^2 binary variables and $6N-6$ linear constraints. Gurobi and CPLEX MIP use a zero objective; CBC maximizes the queen count, which is fixed at N by the row equations and does not change the feasible set. This objective convention is retained with the original CBC measurements.

3.6. Complexity comparison

Table 1 gives asymptotic counts for the complete SAT models. An $N \times N$ board has N rows, N columns, and $O(N)$ diagonals, each with at most N cells.

Here, input GAC means that setting one input of an individual AMO constraint to true makes unit propagation falsify its other inputs. It does not imply complete propagation for the full N-Queens formula. For $N \ge 2$, let $a_E(k)$ and $c_E(k)$ count auxiliary variables and clauses for one k-input AMO, with both zero for $k \le 1$. Exact board-level counts are

$$V_E(N) = N^2 + (2N+2)a_E(N) + 4\sum_{k=2}^{N-1} a_E(k), \tag{24}$$

$$C_E(N) = 2N + (2N+2)c_E(N) + 4\sum_{k=2}^{N-1} c_E(k). \tag{25}$$

The $2N$ term counts row and column ALO clauses. The one-constraint counts for pairwise, Binary, and Sequential are

$$(a_{\text{pair}}, c_{\text{pair}}) = (0, \tbinom{k}{2}), \tag{26}$$

$$(a_{\text{binary}}, c_{\text{binary}}) = (\lceil \log_2 k \rceil, k\lceil \log_2 k \rceil), \tag{27}$$

$$(a_{\text{seq}}, c_{\text{seq}}) = (k-1, 3k-4). \tag{28}$$

Product uses pairwise AMO for $k < 6$. For larger k, let $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. Its counts are

$$(a, c) = \left(p+q,\ 2k + \binom{p}{2} + \binom{q}{2} + pq - k \right). \tag{29}$$

For Commander, groups have sizes $t_j \le 3$ and $m = \lceil k/3 \rceil$. When $k > 3$, the recursive counts are

$$a(k) = m + a(m), \qquad c(k) = \sum_j \binom{t_j}{2} + k + m + c(m). \tag{30}$$

The pairwise case is the recursion base. The m reverse clauses are included in the count. These expressions count generated clauses before solver preprocessing.

Table 2
Generated SAT formula sizes; the timed-out Sequential $N = 200$ cell uses exact formula counts

Encoding	$N = 100$		$N = 200$	
	Variables	Clauses	Variables	Clauses
Binomial	10,000	1,646,900	40,000	13,253,800
Binary	13,678	269,024	48,564	1,236,200
Commander	30,536	138,628	120,882	555,980
Sequential	49,402	117,812	198,802	475,612
Product	19,428	117,736	66,970	482,064

4. Experimental results and comparison

4.1. Protocol and verification

The principal comparison uses $N \in \{20, 50, 100, 150, 200\}$ for all ten methods. Each method-size cell has three repetitions, one solver worker, and a 120-s deadline for the full child process, including startup and validation. The experiments ran on one Windows 11 machine; the current hardware check identifies an AMD Ryzen 7 8845H, 16 logical processors, and 27.8 GiB of usable physical memory. Recorded package versions are Python 3.14.4, python-sat 1.9.dev15, PuLP 3.3.2, OR-Tools 9.15.6755, Gurobi 13.0.3, DOcplex 2.32.264, and the CPLEX Python distribution 22.2.0.1 with the Standard 22.2 native runtime.

SAT, CBC, and CP-SAT measurements come from two sessions with identical Python source. The CPLEX results come from a later licensed session. Gurobi was measured again after separating environment initialization and omitting redundant singleton-diagonal inequalities. Each table cell selects one session; repetitions are never pooled across sessions. Metadata retain source fingerprints, package versions, and the licensed IBM binary hashes. Table 3 includes all 150 repetitions at the five common sizes: 141 successful runs and nine deadlines. A median is shown only when all three repetitions complete successfully.

The primary metric is $t_{\text{total}} = t_{\text{encoding}} + t_{\text{solving}}$, where encoding builds the CNF or exact model and solving measures the solver invocation. SAT invocation includes Glucose initialization, clause loading, solving, model retrieval, and solver disposal. Backend-specific API initialization performed inside an invocation is included there. Explicit Gurobi environment/license initialization is recorded as t_{setup} and excluded from t_{total}; its $N = 200$ median is 0.006 s. Separate wall clocks include placement extraction inside the worker and startup plus validation in the parent. These metrics describe different boundaries and are not end-to-end equivalents. Medians are computed for each metric independently.

All returned placements were validated before a run was marked SAT. The public validation script checks exhaustive AMO input assignments through $n = 8$, input unit propagation and exact AMO counts through $n = 64$, and exact board counts through $N = 12$. It compares every projected SAT placement through $N = 8$ with independent permutation enumeration, obtaining counts $1, 0, 0, 2, 10, 4, 40, 92$. The revised runner retains per-run placements in JSONL; all 15 corrected Gurobi witnesses were revalidated from disk. Earlier sessions did not preserve their placements, so those witnesses cannot be checked retrospectively from the CSVs.

Integration checks additionally exercise all ten methods at $N \in \{1, 2, 3, 4, 8\}$, including the two infeasible sizes. Commercial checks require their installed runtimes and licenses. These checks and subsequent integration fixes do not replace the recorded benchmark observations.

4.2. Formula size and SAT performance

Table 2 reports generated sizes before preprocessing. The Sequential $N = 200$ size is derived from the verified count expressions because the corresponding runs reached the process deadline. Pairwise uses no auxiliary variables but grows cubically in clauses; the other models reduce clause growth at the cost of auxiliary structure.

At $N = 100$, Binary has the lowest SAT median, 0.534 s, although its formula has more clauses than Commander or Product. Product is lowest at $N = 150$ (1.162 s); Commander is lowest at $N = 200$ (2.252 s). Sequential requires 108.028 s at $N = 150$ and reaches the deadline at $N = 200$. Formula size alone therefore does not explain the observed ordering. Figure 1 separates solver invocation time from construction cost.

Table 3
Median construction plus solver invocation time (s), three repetitions per cell. All SAT rows use Glucose4; Timeout means three 120-s process deadlines

Method	$N = 20$	$N = 50$	$N = 100$	$N = 150$	$N = 200$
Binomial	0.025	0.367	3.237	10.397	24.805
Binary	0.300	0.131	0.534	1.519	3.208
Commander	0.019	0.133	0.708	1.253	2.252
Sequential	0.036	3.000	71.145	108.028	Timeout
Product	0.581	28.036	36.796	1.162	2.806
CBC	Timeout	0.771	9.805	62.080	Timeout
Gurobi	0.010	0.168	0.125	0.316	0.474
CPLEX MIP	0.034	0.118	0.335	0.719	1.467
CP Optimizer	0.050	0.029	0.052	0.060	0.064
CP-SAT	0.044	0.311	1.740	12.252	44.776

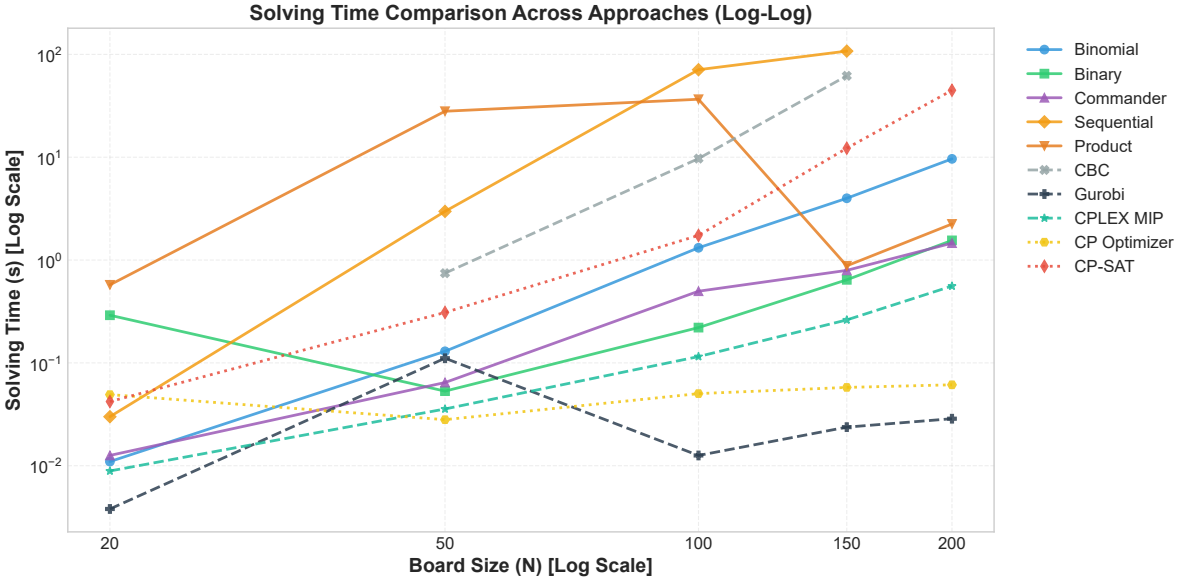


Figure 1: Median solver invocation time at the five common board sizes; both axes are logarithmic. Deadline cells have no completion-time point. Lines connect successful measured sizes and do not imply monotone scaling.

4.3. Comparison with exact baselines

At $N = 200$, CP Optimizer records a median of 0.064 s, compared with 0.474 s for corrected Gurobi, 1.467 s for CPLEX MIP, 2.252 s for Commander/Glucose4, and 44.776 s for CP-SAT. CBC reaches the deadline at $N = 20$ and $N = 200$ despite completing the intervening sizes; these outcomes are timeouts, not infeasibility results. The CP models use N column variables and three `AllDifferent` constraints, whereas the ILP models use N^2 Boolean variables. This difference in formulation is part of the comparison.

Figure 2 compares exact baselines with the lowest measured SAT median at each size. That SAT series is selected retrospectively among five encodings and is not one fixed method. The same ten methods are attempted at every size, unlike the unequal coverage of the archive.

The cactus plot in Figure 3 sorts each method's successful size-level medians. Every method attempts the same five board sizes. Sequential ends at four successes and CBC at three; a cutoff duration is not inserted as a solved runtime. Three repetitions per size are used to estimate a median, not treated as independent problem instances.

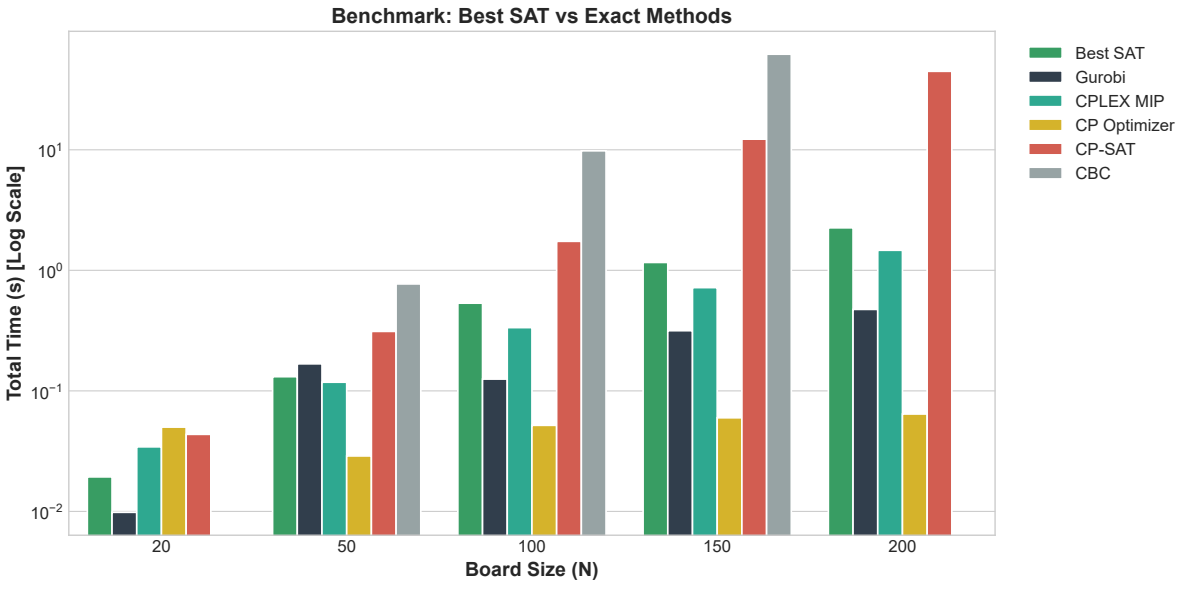


Figure 2: Median construction plus solver invocation time at common sizes, with a logarithmic time axis. Best SAT selects the lowest SAT median per size; omitted CBC bars are deadline cells. Measurements retain separate session provenance.

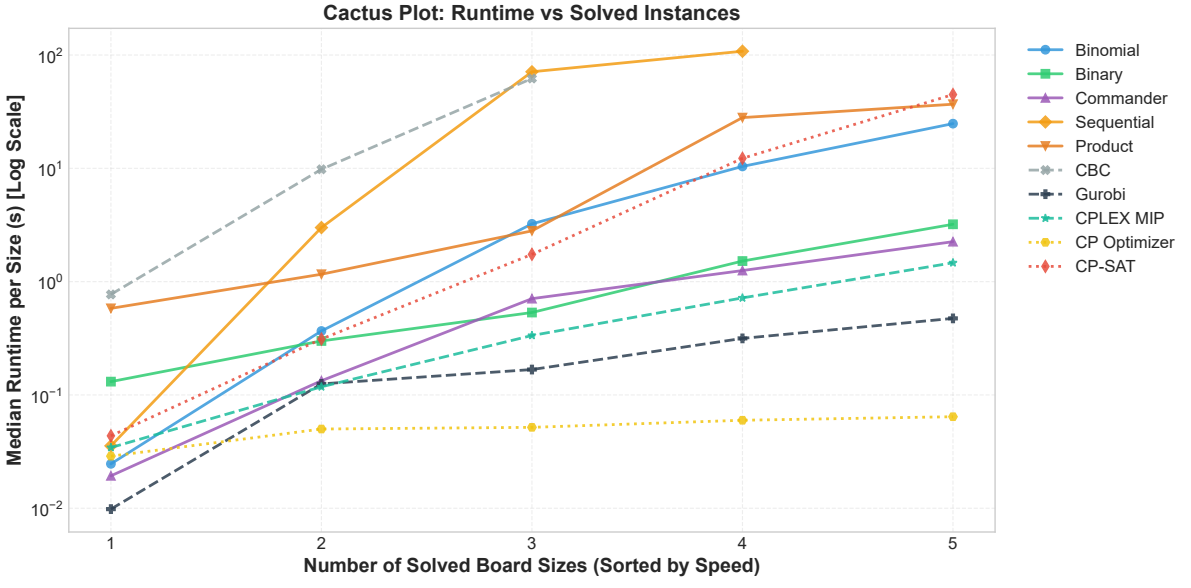


Figure 3: Cactus plot of sorted median construction plus solver invocation times over five common board sizes. The ordinate is time per size, not cumulative time; shorter curves indicate recorded deadlines.

4.4. Search behavior, variation, and limits

Table 4 relates runtime to the search counters retained in the raw logs. At $N = 100$, Sequential has fewer clauses than Binary but records substantially more conflicts and propagations. Product has a larger formula at $N = 150$ than at $N = 100$, yet conflicts fall from 100,361 to 1,883 and propagations from 38,098,586 to 800,065. The accompanying runtime decrease illustrates non-monotone search effort across these board sizes; formula size alone does not predict runtime. These counters describe the observed runs, rather than identify the causal effect of a particular heuristic.

Table 4
Selected SAT search counters and median construction plus invocation times from the recorded sessions

Encoding	N	Total time (s)	Conflicts	Propagations
Binary	100	0.534	346	123,320
Sequential	100	71.145	101,044	50,310,102
Product	100	36.796	100,361	38,098,586
Product	150	1.162	1,883	800,065

Table 5
Selected runtime ranges over three repetitions (s). Min, median, and max refer to construction plus invocation; the final column includes the full child-process lifecycle

Method	N	Min	Median	Max	Process median
Gurobi	20	0.0084	0.0098	0.0397	2.3364
CP Optimizer	100	0.0460	0.0518	0.0785	2.1677
CP Optimizer	200	0.0589	0.0643	0.0760	2.4855
Commander	200	2.2219	2.2524	2.3576	4.7967

For all 24 SAT method-size cells that completed three repetitions, the four recorded search counters (conflicts, decisions, propagations, and restarts) are identical across repetitions. These repeats therefore describe timing variation for the same observed search statistics, rather than three independent instances or a seeded search study.

Table 5 shows why the metric boundary matters. At $N = 200$, CP Optimizer and Commander have construction-plus-invocation medians of 0.064 and 2.252 s, but full-process medians of 2.485 and 4.797 s. Their ordering is unchanged, while their relative difference is much smaller when startup and shutdown are included. At $N = 20$, the three Gurobi totals range from 0.0084 to 0.0397 s; the 0.0098-s median alone hides this variation. The repository provides corresponding ranges for all common-size cells.

All five isolated AMO encodings are GAC on their original inputs, as established by the clauses and public tests. This property does not make the complete board formula propagation complete. Binary and ternary clauses expose implications cheaply, but propagation paths, decisions, and learned clauses still depend on the encoding and solver state [1, 12]. Short original clauses do not guarantee short learned clauses, and a long implication chain need not span many decision levels. The run records include conflicts, decisions, propagations, and restarts, but no LBD distributions; they cannot establish which CDCL mechanism caused a runtime difference.

The experiment uses one machine, engine-default search settings apart from the worker count and deadline, and separate measurement sessions. It has no seeded search ablation, random method-order blocking, independent machines, or significance test. Observed min–max ranges in the raw data are descriptive, not confidence intervals. Empty-board N-Queens admits direct polynomial constructions; five feasible sizes up to $N = 200$ assess these implementations but do not establish asymptotic performance or behavior on harder completion instances [6].

The supplementary benchmarks retain the 116-row archive, an earlier 45-run protocol check, all 78 licensed IBM runs over 13 sizes, and the three-run Product $N = 75$ supplement. The latter has a median of 31.846 s. A coverage index maps all 130 method-size cells to their source and status. These additional records extend coverage but are not pooled into the common-size comparison. Archived rows lack individual repetitions and complete provenance, so the main figures use the repeated common-size data instead.

5. Conclusion and future work

Five established AMO encodings and five exact baselines were evaluated on the same five empty-board sizes. Commander has the lowest SAT median at $N = 200$, while the licensed CP Optimizer session records a lower median than the SAT and ILP sessions. Sequential and CBC reach the deadline at that size. Exact counts and public semantic tests support the implementation; timing results remain descriptive because of the limited instance set and separate sessions.

The most useful follow-ups are a controlled symmetry-breaking ablation and experiments on feasible and infeasible N-Queens Completion instances. Additional SAT backends or learned-clause measurements should answer a stated experimental question under the same timing protocol, rather than enlarge the comparison without adequate repetitions.

CRediT authorship contribution statement

Vu Minh Hoang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies

The author used generative AI tools to assist with code revisions, figure design, and LaTeX formatting. The author reviewed the resulting code, figures, and text and remains responsible for the manuscript.

Data availability

The source code, benchmark data, and reproduction scripts are available at https://github.com/Ohhalo02/nqueens-sat-optimization-01. The repository contains raw CSVs, summaries, metadata, and JSONL placements under `nqueens_sat/results/benchmarks/`. The `comparison.csv` dataset selects one benchmark source per common-size cell, with corresponding file manifests and verification scripts to regenerate all manuscript tables and figures. Current artifact checks normalize text line endings to LF before hashing; original session metadata and their collection-time fingerprints remain unchanged.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI-09), pp. 399–404.

[2] Audemard, G., Simon, L., 2018. Glucose in the SAT 2018 competition, in: Proceedings of SAT Competition 2018: Solver and Benchmark Descriptions, pp. 18–19. URL: https://helda.helsinki.fi/handle/10138/237063.

[3] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31. URL: https://doi.org/10.1016/j.disc.2007.12.043.

[4] Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–9. URL: https://archive.modref.org/files/papers/2010/ModRef2010-11-A-New-SAT-Encoding-of-the-At-Most-One-Constraint.pdf.

[5] Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most-k constraint: Some old, some new, some fast, some slow, in: Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–15. URL: https://archive.modref.org/files/papers/2010/ModRef2010-13-SAT-Encodings-of-the-At-Most-k-Constraint.pdf.

[6] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-Queens Completion. Journal of Artificial Intelligence Research 59, 815–848. URL: https://doi.org/10.1613/jair.5512.

[7] Google Developers, 2019. Or-tools. URL: https://developers.google.com/optimization.

[8] Gurobi Optimization, LLC, 2026. Gurobi optimizer reference manual. URL: https://doc.gurobi.com/projects/optimizer/en/current/. version 13.0; accessed 14 October 2026.

[9] IBM Corporation, 2026. Ibm ilog cplex optimization studio documentation. URL: https://www.ibm.com/docs/en/icos/22.2.0. version 22.2.0; accessed 14 October 2026.

[10] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: International Conference on Theory and Applications of Satisfiability Testing, Springer. pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26.

[11] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects. Fourth Workshop on Constraints in Formal Verification (CFV 2007). URL: https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.

[12] Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. IOS Press. volume 185, pp. 131–153. URL: https://doi.org/10.3233/978-1-58603-929-5-131.

[13] Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521. URL: https://doi.org/10.1109/12.769433.

[14] Mitchell, S., O'Sullivan, M., Dunning, I., 2011. PuLP: A Linear Programming Toolkit for Python. Technical Report. University of Auckland. URL: https://optimization-online.org/wp-content/uploads/2011/09/3178.pdf.

[15] Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S., 2001. Chaff: Engineering an efficient SAT solver, in: Proceedings of the 38th Annual Design Automation Conference, pp. 530–535. URL: https://doi.org/10.1145/378239.379017.

[16] Nemhauser, G.L., Wolsey, L.A., 1988. Integer and Combinatorial Optimization. John Wiley & Sons. URL: https://doi.org/10.1002/9781118627372.

[17] Prestwich, S.D., 2009. CNF encodings, in: Handbook of Satisfiability. IOS Press. volume 185, pp. 75–97. URL: https://doi.org/10.3233/978-1-58603-929-5-75.

[18] Régin, J.C., 1994. A filtering algorithm for constraints of difference in csps, in: Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94), pp. 362–367.

[19] Rossi, F., van Beek, P., Walsh, T. (Eds.), 2006. Handbook of Constraint Programming. Elsevier. URL: https://shop.elsevier.com/books/handbook-of-constraint-programming/rossi/978-0-444-52726-4.

[20] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: International Conference on Principles and Practice of Constraint Programming, Springer. pp. 827–831. URL: https://doi.org/10.1007/11564751_73.